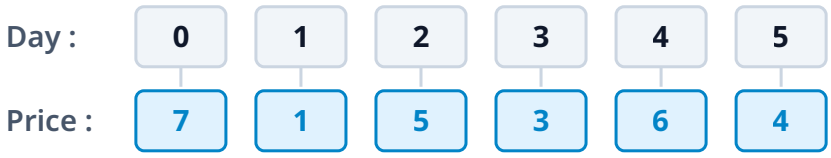
Best Time to Buy and Sell Stock (LeetCode 121)

Problem

You are given an array:

```
prices = [7,1,5,3,6,4]
```

Each index represents a day.



You can

- Buy once
- Sell once
- Buy must happen before Sell

Find the maximum profit.

Brute Force

Think like this:

For every day, *"What if I buy here?"* Then check every future day.

```
for(int buy=0;buy<n;buy++){
    for(int sell=buy+1;sell<n;sell++){
        profit=max(profit,prices[sell]-prices[buy]);
    }
}
```

Time

$O(n^2)$

Works but is slow.

First DP Thought

Whenever you see

- Maximum
- Minimum
- Choices

Start thinking

- Can I define a DP state?

Let's define

`dp[i]`

Meaning

Maximum profit we can make considering days 0...i.

Notice carefully.
We are **NOT** saying

Profit on day i

Instead

Best profit until day i

Let's Fill the DP Table

Example `prices` : 7 1 5 3 6 4

Initially

`dp[0]=0`

Because Day 0 Cannot sell. Profit `0`

Day 1
Price: `1`
Minimum price seen so far: `1`
Profit if we sell today: 1-1=0
Previous maximum: `dp[0]=0`
So `dp[1] =max(0,0) =0`
DP: 0 0

Day 2
Price: `5`
Minimum so far: `1`
Sell today: 5-1=4
Previous best: `0`
So `dp[2] =max(0,4) =4`
DP: 0 0 4

Day 3
Price: `3`
Minimum: `1`
Sell today: 3-1=2
Previous: `4`
So `dp[3] =max(4,2) =4`
DP: 0 0 4 4

Day 4
Price: `6`
Sell: 6-1=5
Previous: `4`
So `dp[4] =max(4,5) =5`
DP: 0 0 4 4 5

Day 5
Sell: 4-1=3
Previous: `5`
So `dp[5] =max(5,3) =5`
Final DP: 0 0 4 4 5 5

Answer

`dp[n-1]=5`

DP Formula

Now observe carefully. How was each state calculated?

`dp[i] =max( dp[i-1], sellToday )`

And `sellToday =prices[i]-minimumPrice`

Therefore

dp[i] = max(dp[i-1], prices[i] - minimumPrice)

This is the DP recurrence.

DP Code

```
class Solution {
public:
    int maxProfit(vector<int>& prices) {
        int n = prices.size();
        vector<int> dp(n, 0);
        int minPrice = prices[0];
        dp[0] = 0;

        for(int i = 1; i < n; i++){
            minPrice = min(minPrice, prices[i]);
            dp[i] = max(dp[i-1], prices[i] - minPrice);
        }

        return dp[n-1];
    }
};
```

Time

O(n)

Space

O(n)

Why do we even need dp[]?

Let's inspect the recurrence.

`dp[i] =max(dp[i-1],sellToday)`

Notice something. To calculate `dp[5]`, do we need `dp[0]`, `dp[1]`, `dp[2]`, `dp[3]` ? **No.**

We only need `dp[4]`. Only the previous state. That means the entire array is unnecessary.

Space Optimization

Instead of `dp[0] dp[1] dp[2] dp[3] dp[4] dp[5]`, keep only `previousAnswer`. Rename it `maxProfit`.

Old DP

`dp[i]=max(dp[i-1],prices[i]-minPrice);`

Optimized

`maxProfit=max(maxProfit,prices[i]-minPrice);`

Exactly the same recurrence.

Optimized Code

```
class Solution {
public:
    int maxProfit(vector<int>& prices) {
        int minPrice = prices[0];
        int maxProfit = 0;

        for(int i = 1; i < prices.size(); i++){
            minPrice = min(minPrice, prices[i]);
            maxProfit = max(maxProfit, prices[i] - minPrice);
        }

        return maxProfit;
    }
};
```

Time

O(n)

Space

O(1)

Your Doubt: Is this really Dynamic Programming?

You asked: *"How is this DP? Can this be called Dynamic Programming?"*

The answer is yes, but there are two ways to look at it.

View 1: Dynamic Programming

We defined a state:

`dp[i] = Maximum profit till day i`

Transition:

`dp[i] =max(dp[i-1], prices[i]-minimumPrice)`

Since each state depends only on the previous state, we optimize the DP array into one variable. This is called **Space-Optimized DP**.

View 2: Greedy

We can also think differently. At every day:

- Keep track of the minimum price seen so far.
- If selling today gives a better profit than before, update the answer.

This naturally leads to:

```
minPrice=min(minPrice,prices[i]);
maxProfit=max(maxProfit, prices[i]-minPrice);
```

So this same implementation is also a **Greedy algorithm**.

Why can one algorithm be both DP and Greedy?

This is an important concept. Some problems satisfy both:

- The DP recurrence simplifies so much that only one previous state is needed.
- The optimal choice at each step is also the globally optimal choice.

In such cases, the optimized DP and the greedy solution become identical.

How Should You Approach Stock Problems?

This is the mindset that will help you solve all stock problems.

Step 1: Count the transactions.

Ask yourself:

- One transaction?
- Unlimited transactions?
- At most two?
- At most k?
- Cooldown?
- Transaction fee?

These constraints determine the DP state.

Step 2: Define the state.

A common pattern for stock problems is:

`dp[day][buy]` or `dp[day][buy][transactionsLeft]`

For example:

`dp[i][1]` means: Maximum profit starting from day i when you are allowed to buy.

`dp[i][0]` means: Maximum profit starting from day i when you are holding a stock and the next action is to sell.

For more advanced versions, you'll extend this to include the number of transactions remaining.

Step 3: Write the choices.

If you can buy:

- Buy OR Skip

If you can sell:

- Sell OR Skip

Each state is the maximum of these choices.

Step 4: Convert Recursion → Memoization → Tabulation → Space Optimization.

This is the standard DP workflow:

Final Takeaway

For LeetCode 121 (Best Time to Buy and Sell Stock):

> The problem can be formulated as Dynamic Programming using the state `dp[i]` = maximum profit until day i.
>
> The DP recurrence is:
>
> ***dp[i] = max(dp[i-1], prices[i] - minPrice)***
>
> Since `dp[i]` depends only on `dp[i-1]`, the DP array can be optimized into a single variable (`maxProfit`), reducing the space from ***O(n)*** to ***O(1)***.
>
> The final optimized solution also has a natural Greedy interpretation because it greedily maintains the minimum buying price and the best profit seen so far.

This is why you'll often see this problem categorized as both Dynamic Programming and Greedy. Understanding the DP derivation is valuable because the more advanced stock problems (multiple transactions, cooldown, transaction fee, etc.) cannot usually be solved with a simple greedy approach—they rely on the same state-based DP thinking you started here.